



OBJECTIFS DE LA SEANCE

À la fin de cette séance, vous serez capable de :

- Créer des routes RESTful avec Express
- Implémenter des controllers pour la logique métier
- Gérer les opérations CRUD complètes
- Créer une API de gestion de films et locations
- Tester l'API avec des outils (Thunder Client/Postman)

PREREQUIS

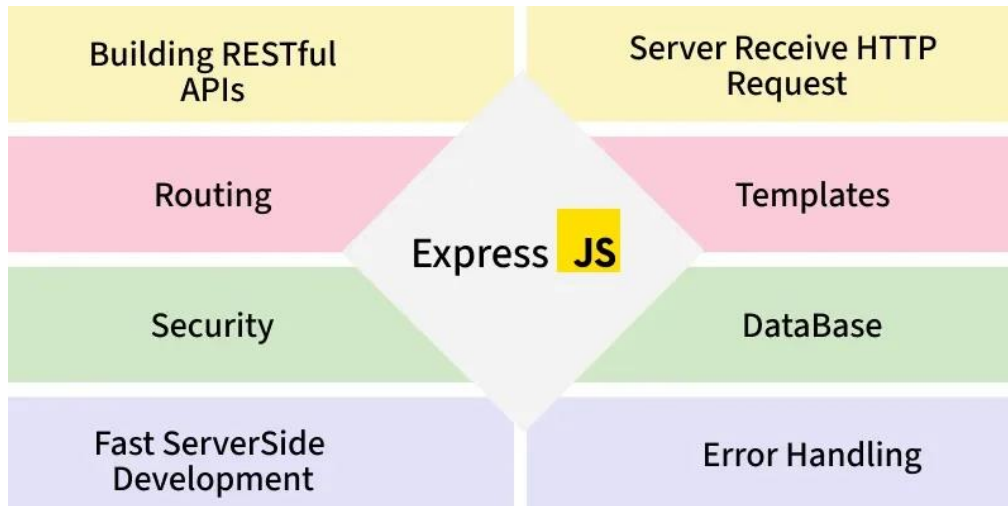
- Séance 7 terminée (modèles Mongoose créés)
- Serveur Express fonctionnel

RESSOURCES

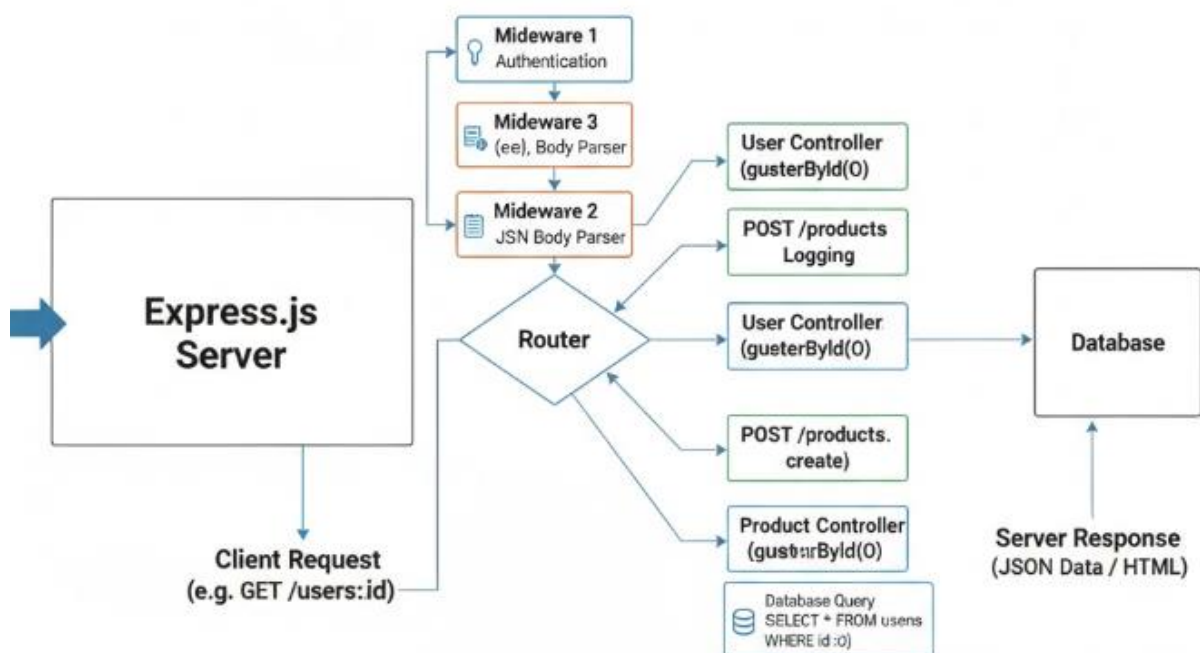
- [Express Routing](#)
- [REST API Best Practices](#)
- [HTTP Status Codes](#)
- [Mongoose Queries](#)
- [Rate Limiting](#)

INTRODUCTION

Express.js est l'un des frameworks les plus populaires pour Node.js, permettant aux développeurs de créer efficacement des applications web et des API. En effet, plus de 70 % des applications web [Node.js](https://nodejs.org/) utilisent Express.js grâce à sa conception minimaliste et puissante.



À mesure que les applications gagnent en complexité, l'organisation du code devient essentielle pour éviter l'encombrement et la confusion. C'est là qu'interviennent les contrôleurs : ils permettent de **structurer** votre application de manière plus claire, plus modulaire et plus facile à maintenir. En séparant la logique de routage de la logique métier principale, les contrôleurs facilitent la mise à l'échelle, les tests.





QU'EST-CE QU'UN CONTROLEUR ?

Dans [Express.js](#), un contrôleur est une fonction JavaScript (ou un ensemble de fonctions) qui gère la manière dont votre application répond aux requêtes HTTP entrantes. Il sert d'interface entre les définitions de routes et la logique métier ou les opérations de base de données. En d'autres termes, tandis que la route détermine l'URL ou la méthode à écouter, les contrôleurs décident du traitement de la requête et de la réponse à renvoyer. Cette séparation permet de conserver un code propre, organisé et plus facile à gérer.

POURQUOI UTILISER UN CONTROLEUR ?

À mesure que votre application Express.js se développe, gérer toute la logique directement dans vos fichiers peut rapidement devenir complexe et difficile à maintenir. C'est là que les contrôleurs entrent en jeu. Ils vous permettent de séparer la logique de routage de la logique métier, rendant ainsi votre code plus clair, mieux organisé et plus facile à gérer. Grâce aux contrôleurs, vous pouvez structurer votre projet de manière modulaire, ce qui améliore la lisibilité, favorise la réutilisation et permet à votre application de s'adapter efficacement à la charge.

- Séparation des préoccupations : elle permet de garder les définitions de routes propres en déplaçant la logique dans des fichiers séparés.
- Modularité : Elle rend votre code réutilisable et testable.
- Maintenabilité : Il est plus facile de mettre à jour ou de remanier la logique lorsqu'elle n'est pas étroitement liée aux routes.
- Évolutivité : La gestion est plus facile à mesure que l'application se développe.



PARTIE 1 : ETAPES POUR CREER ET UTILISER UN CONTROLEUR

1. CREER LE FICHIER DU CONTROLEUR

```
// controllers/userController.js

// Logique pour récupérer un utilisateur
exports.getUserProfile = (req, res) => {
  const userId = req.params.id;
  // Logique métier (ex: appel base de données) ici...
  res.status(200).json({
    id: userId,
    name: "Marc"
  });
};

// Logique pour créer un utilisateur
exports.createUser = (req, res) => {
  res.status(201).json({ message: "Utilisateur créé !" });
};
```

2. LIER LE CONTROLEUR AU ROUTEUR

```
// routes/userRoutes.js
const express = require('express');
const router = express.Router();
const userCtrl = require('../controllers/userController');

// On définit la route et on passe la fonction du contrôleur
router.get('/:id', userCtrl.getUserProfile);
router.post('/', userCtrl.createUser);

module.exports = router;
```

3. UTILISER LE ROUTEUR DANS L'APPLICATION

```
// app.js
const express = require('express');
const app = express();
const userRoutes = require('./routes/userRoutes');

app.use(express.json()); // Middleware pour lire le JSON

// Toutes les routes dans userRoutes commenceront par /api/users
app.use('/api/users', userRoutes);

app.listen(3000, () => console.log("Serveur lancé sur le port 3000"));
```



PARTIE 2 : PRINCIPES REST ET ROUTES (20 MIN)

2.1 ARCHITECTURE REST

REST → Representational State Transfer:

- Architecture pour créer des API
- Utilise les méthodes HTTP (GET, POST, PUT, DELETE)
- URLs représentent des ressources
- Stateless (sans état)

2.2 CONVENTION DES ROUTES

Méthode	Route	Action (Controller)	Description
GET	/api/movies	index	Récupère la liste complète des films.
GET	/api/movies/:id	show	Récupère les détails d'un film spécifique via son ID.
POST	/api/movies	create	Ajoute un nouveau film dans la base MongoDB.
PUT	/api/movies/:id	update	Met à jour l'intégralité des infos d'un film existant.
DELETE	/api/movies/:id	destroy	Supprime définitivement un film de la base de données.



2.3 CODES DE STATUT HTTP

Catégorie	Code	Statut	Description
Succès (2xx)	200	OK	La requête a réussi.
	201	Created	La ressource a été créée avec succès.
	204	No Content	Requête réussie, mais il n'y a rien à renvoyer.
Erreurs Client (4xx)	400	Bad Request	La syntaxe de la requête est erronée.
	401	Unauthorized	Authentification nécessaire (qui êtes-vous ?).
	403	Forbidden	Droits insuffisants (vous n'avez pas le droit d'être ici).
	404	Not Found	La ressource demandée n'existe pas.
Erreurs Serveur (5xx)	500	Internal Error	Le serveur a rencontré un problème imprévu.
	503	Service Unavailable	Le serveur est temporairement en maintenance ou surchargé.



2.4 FORMAT DE REPONSE JSON

```
1 // Format de succès
2 {
3   "success": true,
4   "data": { /* ... */ },
5   "message": "Opération réussie"
6 }
7
8 // Format d'erreur
9 {
10  "success": false,
11  "message": "Message d'erreur",
12  "errors": ["détail 1", "détail 2"]
13 }
14
15 // Format de liste avec pagination
16 {
17   "success": true,
18   "count": 20,
19   "total": 100,
20   "page": 1,
21   "totalPages": 5,
22   "data": [ /* ... */ ]
23 }
```



EXERCICE 1 : MOVIE CONTROLLER (40 MIN)

CREER LE CONTROLLER

- Créez le fichier `src/controllers/movie.controller.js` :

```
import Movie from '../models/Movie.js';

// @desc    Obtenir tous les films
// @route    GET /api/movies
// @access   Public
> export const getAllMovies = async (req, res, next) => { ...
};

// @desc    Obtenir un film par ID
// @route    GET /api/movies/:id
// @access   Public
> export const getMovieById = async (req, res, next) => { ...
};

// @desc    Créer un nouveau film
// @route    POST /api/movies
// @access   Private/Admin
> export const createMovie = async (req, res, next) => { ...
};

// @desc    Modifier un film
// @route    PUT /api/movies/:id
// @access   Private/Admin
> export const updateMovie = async (req, res, next) => { ...
};

// @desc    Supprimer un film
// @route    DELETE /api/movies/:id
// @access   Private/Admin
> export const deleteMovie = async (req, res, next) => { ...
};

// @desc    Obtenir les statistiques des films
// @route    GET /api/movies/stats
// @access   Private/Admin
> export const getMovieStats = async (req, res, next) => { ...
};
```

NB : A ce stade , n'implémentez pas les fonctions , définissez-les justes, et affichez un message dans la console

CREER LES ROUTES

- Créez le fichier `src/routes/movie.routes.js` :

```
import express from 'express';
import {
  getAllMovies,
  getMovieById,
  createMovie,
  updateMovie,
```



```

deleteMovie,
getMovieStats,
getSimilarMovies
} from '../controllers/movie.controller.js';
// import { protect, admin } from '../middleware/auth.middleware.js'; // Séance 9

const router = express.Router();

// Routes publiques
router.get('/', getAllMovies);
router.get('/stats', getMovieStats); // TODO: Protéger avec admin (séance 9)
router.get('/:id', getMovieById);
router.get('/:id/similar', getSimilarMovies);

// Routes protégées admin (sera activé séance 9)
// router.post('/', protect, admin, createMovie);
// router.put('/:id', protect, admin, updateMovie);
// router.delete('/:id', protect, admin, deleteMovie);

// Routes temporaires sans authentification (pour tester)
router.post('/', createMovie);
router.put('/:id', updateMovie);
router.delete('/:id', deleteMovie);

export default router;

```

INTEGRER LES ROUTES DANS LE SERVEUR

- Modifiez le fichier src/server.js :

```

// Importer de la route
import movieRoutes from './routes/movie.routes.js';
// Utilisation de la Routes API
app.use('/api/movies', movieRoutes);

```



PARTIE 3 : LES FONCTIONS MIDDLEWARE

Une fonction middleware dans Express est littéralement une fonction qui se situe "au milieu" (middle) du cycle Requête → Réponse.

3.1 ⚙️ COMMENT ÇA FONCTIONNE ?

Une fonction middleware a accès à trois arguments :

1. **req** : L'objet de la requête (infos envoyées par le client).
2. **res** : L'objet de la réponse (ce qu'on renvoie).
3. **next** : Une fonction qui, une fois appelée, passe le relais au middleware suivant.

3.2 🛠️ LES 4 ROLES PRINCIPAUX

Rôle	Exemple concret
Exécuter du code	Logger l'heure à laquelle une requête arrive.
Modifier req ou res	Extraire l'ID d'un utilisateur à partir d'un token et l'ajouter à req.user.
Terminer le cycle	Envoyer une erreur 403 si l'utilisateur n'est pas autorisé.
Appeler le suivant	Utiliser next() pour avancer dans la chaîne.

Exemple d'un middleware de vérification d'une clef API

```
const checkApiKey = (req, res, next) => {
  const apiKey = req.query.key;

  if (apiKey === '12345') {
    console.log("Accès autorisé ✅");
    next(); // On passe au middleware ou contrôleur suivant
  } else {
    res.status(403).json({ message: "Clé API invalide ❌" });
    // Ici on n'appelle PAS next(), le cycle s'arrête.
  }
};

// Utilisation sur une route spécifique
app.get('/api/data', checkApiKey, (req, res) => {
  res.send("Voici les données secrètes");
});
```

3.3 🎯 LES 3 TYPES DE MIDDLEWARES

1. **Niveau Application** : Liés à `app.use()` (s'exécutent pour toutes les routes).
2. **Niveau Routeur** : Liés à une route spécifique (comme l'exemple ci-dessus).
3. **Intégrés / Tiers** : Ex: `express.json()` pour lire le corps des requêtes ou `cors()` pour la sécurité.

⚠️ **Attention** : Si on n'appelle pas `next()` et que l'on n'envoie pas de réponse (`res.send()`), le serveur restera "suspendu" et le navigateur du client chargera dans le vide indéfiniment.



3.4 📁 L'OBJET REQUETE (REQ)

Propriété	Source	Usage typique	Exemple d'accès
req.params	URL (Route)	Récupérer un ID spécifique dans l'URL.	/api/movies/:id → req.params.id
req.query	URL (Après le ?)	Filtrage, recherche ou pagination.	?genre=Action → req.query.genre
req.body	Corps (JSON)	Données d'un formulaire ou d'un objet (POST/PUT).	req.body.title
req.headers	En-têtes	Jetons d'authentification ou type de contenu.	req.headers.authorization
req.method	HTTP	Connaître le verbe utilisé (GET, POST...).	req.method (renvoie "POST")

Pour une requête GET /api/movies/123?sort=desc avec un header Authorization: Bearer ABC

- **req.params** sera { id: "123" }
- **req.query** sera { sort: "desc" }
- **req.headers.authorization** sera "Bearer ABC"

⚠ **Attention** : Pour que **req.body** fonctionne (ne soit pas undefined), n'oubliez jamais d'ajouter cette ligne dans le fichier principal (app.js) :

```
app.use(express.json()); // Permet de lire les données JSON envoyées dans le body
```



3.5 🏠 L'OBJET RÉPONSE (RES)

Méthode	Usage	Exemple d'utilisation
res.status()	Définit le code de statut HTTP.	res.status(404)
res.json()	Envoie une réponse au format JSON (le plus utilisé).	res.json({ success: true })
res.send()	Envoie une réponse texte, HTML ou un buffer.	res.send("Bienvenue !")
res.status(x).json()	Combo gagnant : définit le statut et envoie le JSON.	res.status(201).json(data)
res.redirect()	Redirige le client vers une autre URL.	res.redirect('/login')
res.cookie()	Définit un cookie dans le navigateur du client.	res.cookie('token', myToken)

💡 LES BONNES PRATIQUES A RETENIR

1. **L'enchaînement (Chaining)** : On peut enchaîner les méthodes.

```
res.status(200).json(movies);
```

2. **Une seule réponse par requête** : On ne peut envoyer qu'une seule fois une réponse par cycle. Si vous essayer d'appeler `res.json()` deux fois dans la même fonction, Express affichera l'erreur : *"Error: Cannot set headers after they are sent to the client"*.
3. **Le format standard** : Il est conseillé de toujours renvoyer un objet structuré, même en cas d'erreur :

```
res.status(400).json({
  success: false,
  message: "Explication de l'erreur"
});
```

EXERCICE 1 (SUITE) : MOVIECONTROLLER

Maintenant vous allez écrire les instructions permettant d'intercepter, via les routes, les paramètres envoyés au serveur

ROUTES ET CONTROLEURS

Type/Verbe	EndPoint	Action	url
GET	/	getAllMovies	/api/movies/ → tous les films /api/movies/?search=in → les films dont le titre ou la description contiennent « in » /api/movies/?genre=action → Les films d'actions /api/movies/?year=2010 → les films sortis en 2010 /api/movies/?search=in&sort=rating → recherche + tri
GET	/xxx	getMovieById	/api/movies/6992d → film dont l'id est 6992d
GET	/xxx/similar	getSimilarMovies	/api/movies/6992d/similar → les films qui ont le même genre que le film dont l'id est 6992d
GET	/stats	getMovieById	/api/movies/stats
DELETE	/ xxx	deleteMovie	/api/movies/6992d → Suppression du film dont l'id est 6992d
POST	/	createMovie	/api/movies/ → Création d'un film via un <u>formulaire</u>
PUT	/xxx	updateMovie	/api/movies/6992d → Mise à jour des données du films dont l'id est 6992d, via un <u>formulaire</u>



DETAIL DES CONTROLEURS

GETALLMOVIES

- Récupérez les paramètres de requête :

```
//....
// Pagination
const skip = (page - 1) * limit;
// Exécution de la requête
const movies = await Movie.find(query)
  .sort(sortOption)
  .skip(skip)
  .limit(parseInt(limit));

// Comptage total pour la pagination
const total = await Movie.countDocuments(query)
//....
```

Résultat attendu :

```
{ } JSON Preview Visualize
1 {
2   "success": true,
3   "count": 5,
4   "total": 5,
5   "totalPages": 1,
6   "currentPage": 1,
7 >  "data": [ ...
93 ]
94 }
```

GETMOVIEBYID

```
// ...
const movie = await Movie.findById(req.params.id);
// ...
```

```
{ } JSON Preview Visualize
1 {
2   "success": true,
3   "data": {
4     "_id": "6992d85d83b8466815078c30",
5     "title": "Interstellar",
6     "description": "Une équipe d'explorateurs voyage à travers un trou de ver dans l'espace pour assurer la survie de l'humanité.",
7     "poster": "https://image.tmdb.org/t/p/w500/gEU2QniE6E77NI6lCU6Mx1NBvIx.jpg",
8     "backdrop": "https://image.tmdb.org/t/p/original/xu9zaAevzQ5nnrsXN6JcahLnG4i.jpg",
9     "genre": "Science-Fiction",
10    "year": 2014,
11    "duration": 169,
12    "price": 4.99,
13    "rating": 8.6,
14    "isAvailable": true,
15    "rentalCount": 1,
16    "__v": 0,
17    "createdAt": "2026-02-16T08:42:05.168Z",
18    "updatedAt": "2026-02-16T08:42:05.257Z"
19  }
20 }
```



GETMOVIESTATS

```
// ...
const totalRevenue = await Movie.aggregate([
  {
    $group: {
      _id: null,
      total: { $sum: { $multiply: ['$price', '$rentalCount'] } }
    }
  }
]);
// ...
```

```
{ } JSON Preview Visualize
1 {
2   "success": true,
3   "data": {
4     "totalMovies": 5,
5     "estimatedRevenue": 12.97,
6     "byGenre": [
7       {
8         "_id": "Science-Fiction",
9         "count": 2,
10        "avgPrice": 4.49,
11        "avgRating": 8.7,
12        "totalRentals": 2
13      },
14      {
15        "_id": "Action",
16        "count": 1,
17        "avgPrice": 3.99,
18        "avgRating": 9,
19        "totalRentals": 1
20      }
21    ]
22  }
23 }
```

GETSIMILARMOVIES

```
// ...
// Trouver des films du même genre
const similarMovies = await Movie.find({
  genre: { $in: movie.genre },
  _id: { $ne: movie._id }, // Exclure le film actuel
  isAvailable: true
})
.sort({ rating: -1 })
.limit(6);
// ...
```

```
Body Cookies Headers (10) Test Results 200 OK + 41 ms - 928 B Save Response
{ } JSON Preview Visualize
1 {
2   "success": true,
3   "count": 1,
4   "data": [
5     {
6       "_id": "6992d85d83b8466815078c2e",
7       "title": "Inception",
8       "description": "Un voleur qui s'infiltrer dans les rêves des gens pour voler leurs secrets doit accomplir l'inverse : implanter une idée.",
9       "poster": "https://image.tmdb.org/t/p/w500/9gk7adhYeDvHkCSEqAvQNLV5Uge.jpg",
10      "backdrop": "https://image.tmdb.org/t/p/original/s3T8rRG81iav7gFOCNx3H31MoES.jpg",
11      "genre": "Science-Fiction",
12      "year": 2010,
13      "duration": 148,
14      "price": 3.99,
15      "rating": 8.8,
16      "isAvailable": true,
17      "rentalCount": 1,
18      "_v": 0,
19      "createdAt": "2026-02-16T08:42:05.167Z",
```



CREATEMOVIE

```
// ...
// Créer le film
const movie = await Movie.create({
  title,
  description,
  poster,
  backdrop,
  genre,
  year,
  duration,
  price,
  rating
});
// ...
```

UPDATEMOVIE

```
// ...
// Mise à jour
const updatedMovie = await Movie.findByIdAndUpdate(
  req.params.id,
  req.body,
  {
    new: true, // Retourner le document modifié
    runValidators: true // Exécuter les validations
  }
);
// ...
```

DELETEMOVIE

```
// ... Vérifiez qu'il n'y ait pas de locations ...
await movie.deleteOne();
// ...
```



PARTIE 4 : TESTER L'API (30 MIN)

4.1 AVEC THUNDER CLIENT

1. Ouvrir [Thunder Client](#) dans VS Code (icône éclair dans la sidebar)
2. Créer une requête en cliquant sur « New Request »

1 : Méthode

2 : URI

GET

Query Headers ² Auth Body Tests Pre Run

3 : types de Paramètres

Query Parameters

☐ parameter value

4 : Paramètre de la requête

5 : Réponse

Status: 200 OK Size: 2.68 KB Time: 248 ms

```

1 {
2   "success": true,
3   "count": 5,
4   "total": 5,
5   "totalPages": 1,
6   "currentPage": 1,
7   "data": [
8     {
9       "_id": "6992d85d83b8466815078c2f",
10      "title": "The Dark Knight",
11      "description": "Batman affronte le Joker, un criminel anarchiste qui veut plonger Gotham dans le chaos.",

```

- Baladez-vous dans les différents onglets pour voir les types de paramètres qui peuvent être utiles

NB : les éléments de formulaires sont à traiter avec l'onglet « body/Form-encode »

Query Headers ² Auth **Body** Tests Pre Run

JSON XML Text Form **Form-encode** GraphQL Binary

Form Encoded

☐ name value

Import

- Pensez à enregistrer vos requêtes pour pouvoir y revenir

4.2 AVEC POSTMAN

[Postman](#) est un outil de développement permettant de tester, documenter et automatiser des requêtes HTTP vers une API. Il facilite l'envoi de données (req) et l'analyse précise des réponses du serveur (res) sans avoir à coder d'interface front-end.

L'avantage est qu'avec un compte gratuit, vous pouvez sauvegarder vos requêtes en ligne et les organiser en collections

The screenshot shows a Postman interface with a collection named 'R4.10-TP Netflix 2026'. The selected request is a POST to 'localhost:4500/api/movies/'. The body is set to 'x-www-form-urlencoded'. The response is a 500 Internal Server Error with the following JSON body:

```

{
  "success": false,
  "message": "Cannot destructure property 'title' of 'req.body' as it is unde",
  "stack": "TypeError: Cannot destructure property 'title' of 'req.body' as i
    createMovie (file:///Users/david/Library/Mobile%20Documents/com%7Eapple
  
```

Pour envoyer le tableau des genres vous pouvez utiliser :

- le mode formulaire « form-urlencoded » et envoyer la liste des genres
 - Key** : genre[0] | **Value** : Action
 - Key** : genre[1] | **Value** : Drame

The screenshot shows a Postman interface with a collection named 'R4.10-TP Netflix 2026'. The selected request is a POST to 'localhost:4500/api/movies/'. The body is set to 'x-www-form-urlencoded'. The response is a 500 Internal Server Error with the following JSON body:

Key	Value
year	2026
description	Du monde de Westeros nous vient une ...
backdrop	https://media.themoviedb.org/t/p/w192...
poster	https://media.themoviedb.org/t/p/w300...
genre[0]	Action
genre[1]	Science-Fiction
genre[2]	Fantastique
genre[3]	Aventure
duration	45



- Ou utiliser une structure json, via l'option « raw »





EXERCICE 2 : RENTAL CONTROLLER (30 MIN)

- Créez le fichier `src/controllers/rental.controller.js` :

```

1  import Rental from "../models/Rental.js";
2  import Movie from "../models/Movie.js";
3  import User from "../models/User.js";
4
5  // @desc    Louer un film
6  // @route    POST /api/rentals
7  // @access   Private
8  > export const createRental = async (req, res, next) => { ...
66  };
67
68  // @desc    Obtenir les locations d'un utilisateur
69  // @route    GET /api/rentals/my-rentals
70  // @access   Private
71  > export const getMyRentals = async (req, res, next) => { ...
97  };
98
99  // @desc    Obtenir toutes les locations (admin)
100 // @route    GET /api/rentals
101 // @access   Private/Admin
102 > export const getAllRentals = async (req, res, next) => { ...
133  };
134
135 // @desc    Annuler une location
136 // @route    DELETE /api/rentals/:id
137 // @access   Private
138 > export const cancelRental = async (req, res, next) => { ...
176  };
177
178 // @desc    Obtenir les statistiques des locations
179 // @route    GET /api/rentals/stats
180 // @access   Private/Admin
181 > export const getRentalStats = async (req, res, next) => { ...
275  };
276

```

NB : regardez la fonction [populate](#) ,qui permet de récupérer l'entité liée dans une relation



- Créez le fichier `src/routes/rental.routes.js` pour gérer les routes :

Type/Verbe	Endpoints	Controleur	url
GET	/	<code>getAllRentals</code>	<code>/api/rentals/</code> ➔ Toutes les locations
GET	<code>/my-rentals</code>	<code>getMyRentals</code>	<code>/api/rentals/my-rentals</code> ➔ Toutes les locations de l'utilisateur connecté <i>(forcé pour l'instant)</i> <code>/api/rentals/my-rentals&status=active</code> ➔ Toutes les locations en cours de l'utilisateur connecté
GET	<code>/stats</code>	<code>getRentalStats</code>	<code>/api/rentals/stats</code> ➔ Toutes les locations de Stat sur les locations
POST	/	<code>createRental</code>	Création d'une location pour l'utilisateur connecté <i>(forcé pour l'instant)</i>
DELETE	<code>/xxx</code>	<code>cancelRental</code>	<code>/api/rentals/xxx</code> ➔ Supprimer la location dont l'id est xxx

- Intégrez le routeur à l'application

**BONUS : SYSTEME DE RECOMMANDATIONS (20 MIN)****OBJECTIF**

Créer une route qui recommande des films basée sur l'historique de location.

INSTRUCTIONS

- Dans `rental.controller.js`, ajouter le nouveau contrôleur :

```
// @desc  Obtenir des recommandations personnalisées
// @route  GET /api/rentals/recommendations
// @access Private
export const getRecommendations = async (req, res, next) => {

  // 1. Obtenir les genres des films loués par l'utilisateur
  // Si pas d'historique, recommander les plus populaires

  // 2. Compter les genres préférés

  // 3. Trier les genres par préférence

  // 4. Obtenir les IDs des films déjà loués

  // 5. Recommander des films de ces genres non encore loués
};
```

- Ajoutez la route au routeur

**CHECKLIST DE FIN DE SEANCE**

- Movie controller complet (CRUD)
- Rental controller complet
- Routes configurées et testées
- Filtres et recherche fonctionnels
- Tests Client réussis
- Code committé



À RETENIR

PRINCIPES REST

- **Stateless** : Chaque requête est indépendante
- **Ressources** : URLs représentent des ressources
- **Méthodes HTTP** : GET (lire), POST (créer), PUT (modifier), DELETE (supprimer)
- **Codes de statut** : 2xx succès, 4xx erreur client, 5xx erreur serveur

STRUCTURE CONTROLLER

```
// Toujours la même structure
export const actionName = async (req, res, next) => {
  try {
    // 1. Extraire les données de req
    const { param } = req.params;
    const { query } = req.query;
    const { data } = req.body;

    // 2. Validation

    // 3. Logique métier (DB operations)

    // 4. Réponse
    res.status(200).json({
      success: true,
      data: result
    });

  } catch (error) {
    next(error); // Passer à l'error handler
  }
};
```

BONNES PRATIQUES API

- Toujours retourner un JSON avec **success**
- Utiliser les bons codes HTTP
- Valider les données d'entrée
- Gérer les erreurs proprement
- Paginer les listes longues
- Documenter les endpoints
- Versioning de l'API (/api/v1/)



METHODES MONGOOSE UTILES

```
Model.find()           // Trouver plusieurs
Model.findOne()        // Trouver un
Model.findById()       // Par ID
Model.create()         // Créer
Model.findByIdAndUpdate() // Modifier
Model.findByIdAndDelete() // Supprimer
Model.countDocuments() // Compter
Model.aggregate()      // Requêtes complexes
```